

Question #1: A film studio needs a database to track its **Films**, the **Actors** they hire, and the **Production Companies** they work with.

Business Rules:

1. **Films:** Every **Film** has a unique FilmID, a Title, a Budget, and a ReleaseDate.
2. **Actors:** Every **Actor** has a unique ActorID, a Name, and a Phone number.
3. **Casting:** A **Film** casts many **Actors**, and an **Actor** can be cast in many **Films**. The role an actor plays (CharacterName) must be recorded when they are cast in a film.
4. **Production:** Each **Film** is made by exactly one **Director** (who is not tracked separately as an entity). The director's Name is stored directly in the FILM record.
5. **Funding:** A **Film** is funded by one or more **Production Companies** (CompanyID, Name). A company can fund many films.

A. Identify all primary Entities and their corresponding Attributes from the scenario. Underline the Primary Key (PK) for each entity.

B. Identify the three main Relationships (e.g., A relates to B). State the Cardinality (1:1, 1:M, M:N) for each relationship.

C. Rule 3 describes a Many-to-Many (M:N) relationship. Name the Associative Entity (Linking Table) needed to resolve it and list all the attributes it will contain (including the PK).

D. Explain why DirectorName (Rule 4) should NOT be made into a separate entity in this simplified scenario.

A. Entities and Attributes

1. FILM

- **Attributes:**

- FilmID (Primary Key)
- Title
- Budget
- ReleaseDate
- DirectorName

2. ACTOR

- **Attributes:**

- ActorID (Primary Key)
- Name
- Phone

3. PRODUCTION_COMPANY

- **Attributes:**

- CompanyID (Primary Key)
- Name

B.

Relationship	Description	Cardinality
Film–Actor (Casting)	A Film casts many Actors; an Actor can act in many Films.	M:N
Film–Production Company (Funding)	A Film can be funded by many Production Companies; a Company can fund many Films.	M:N
Film–Director	Each Film has exactly one Director (stored as an attribute).	1:1 (embedded, not separate entity)

C. Associative Entities (Linking Tables)**1. Casting**

Used to resolve the M:N relationship between Film and Actor.

Attributes:

- FilmID (FK → Film)
- ActorID (FK → Actor)
- CharacterName
- (Composite PK: FilmID + ActorID)

Associative Entity Name: Casting

Primary Key: (FilmID, ActorID)

2. FilmFunding

Used to resolve the M:N relationship between Film and ProductionCompany.

Attributes:

- FilmID (FK → Film)
- CompanyID (FK → ProductionCompany)

Associative Entity Name: FilmFunding

Primary Key: (FilmID, CompanyID)

D. Why DirectorName should NOT be a separate entity

- The scenario clearly states each film has exactly one director, and the director is not tracked separately.
- Since:
 - There's no requirement to store additional director details (e.g., contact info, past films, etc.),
 - And directors are not shared among films or reused as entities,

Creating a separate Director table would unnecessarily complicate the model.

Therefore, DirectorName is stored directly as an attribute in the Film entity.

Q2:

Question #2: Design an ER Schema for keeping track of information about an Airline Reservation System, given the description below:

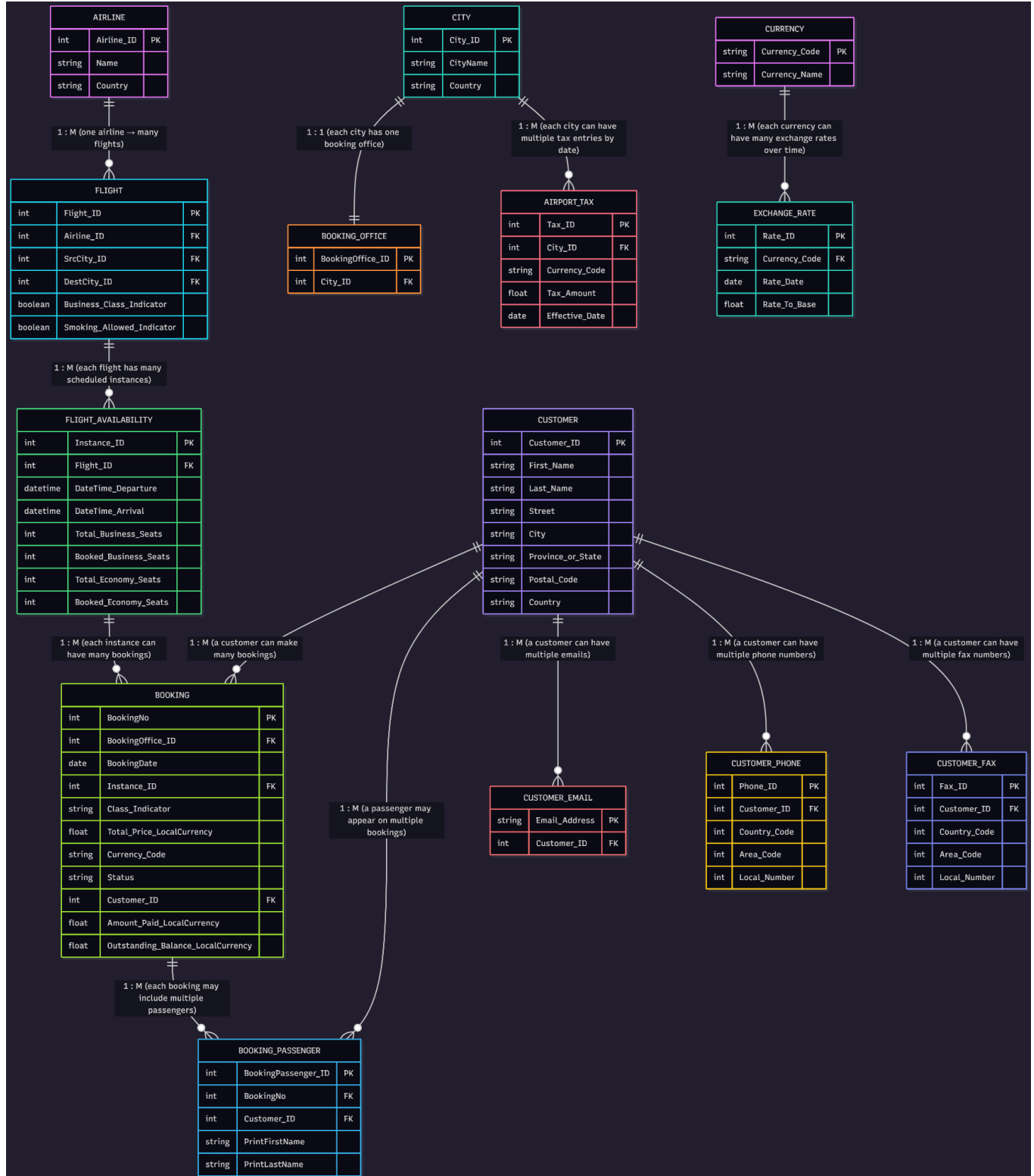
There are 6 different airlines in 6 different countries: Canada – AirCan, USA - USAir, UK - BritAir, France - AirFrance, Germany - LuftAir, Italy - ItalAir. Their flights involve the following 12 cities: Toronto and Montreal in Canada, New York and Chicago in US, London and Edinburgh in UK, Paris and Nice in France, Bonn and Berlin in Germany, Rome and Naples in Italy.

In each of the 12 cities, there is a (single) booking office. You are going to design a central air-reservation database to be used by all booking offices.

The flight has a unique flight number, airline code, business class indicator, smoking allowed indicator. Flight availability has flight number, date + time of departure, number of total seats available in business class, number of booked seats in business class, number of total seats available in economy class, and number of booked seats in economy class.

The customers may come from any country, not just the 6 above, and from any province/state, and from any city. Customer has first & last name, mailing address, zero or more phone numbers, zero or more fax numbers, and zero or more email addresses. Mailing address has street, city, province or state, postal code and country. Phone/fax number has country code, area code and local number. Email address has only one string, and no structure is assumed. A customer can book one or more flights. Two or more customers may have same mailing address and/or same phone number(s) and/or same fax number(s). But the email address is unique for each customer. First and last names do not have to be unique.

Booking has a unique booking number, booking city, booking date, flight number, date + time of departure (in local time, and time is always in hours and minutes), date + time of arrival (in local time), class indicator, total price (airport tax in origin + airport tax in destination + flight price – in local currency. The flightprice for business class is 1.5 times of the listed flight price), status indicator (three types: booked, Canceled – the customer canceled the booking, scratched – the customer had not paid in full 30 days prior to the departure), customer who is responsible for payment, amount-paid-so far (in local currency), outstanding balance (in local currency), the first & last names to be printed on the ticket. The airport taxes must be stored in local currencies (i.e. Canadian dollars, US dollars, British Pounds, French francs, German marks, and Italian Liras). Since the exchange rates change daily, they also must be stored for calculations of all prices involved. Though France, Germany, and Italy have had a common currency for a while, we used the names of their original currencies to involve in this exercise currency exchange rates and their changes.



I have attached the png file for this as well as a handwritten scanned ERD.

AIRLINEAirline_ID PK

Name

Country

CITYCity_ID PK

CityName

Country

BOOKING_OFFICEBookingOffice_ID (PK)

City_ID (FK → CITY)

FLIGHTFlight_ID PK

Airline_ID FK

SrcCity_ID FK

DestCity_ID FK

bussiness_class_indicator (Y/N)

smoking_allowed_indicator (Y/N)

FLIGHT_AVAILABILITY

Flight_ID FK

Instance_ID PK

DateTime_Departure

DateTime_Arrival

no_of_total_economy

no_of_booked_economy

no_of_total_business

no_of_booked_business

CUSTOMERCustomer_ID PK

First_name

Last_name

Mailing_address (COMPOSITE)

street, city, province or state, postal code, country - treat it all as separate attributes.

Phone_no (COMPOSITE) (MULTI-VALUED ATTRIBUTE)

country_code, area_code, local_number

Fax_numbers (COMPOSITE) (MULTI-VALUED ATTRIBUTE)

country_code, area_code, local_number

CUSTOMER_EMAILEmail_address (MULTI-VALUED ATTRIBUTE) PK

Customer_ID FK

CUSTOMER_PHONE

Phone_ID PK

Customer_ID FK

country_code

area_code

local_number

CUSTOMER_FAX

Fax_ID PK

Customer_ID FK

country_code

area_code

local_number

BOOKING

BookingNo (PK)

BookingOfficeID (FK → BOOKING_OFFICE) booking city/office

BookingDate (date)

InstanceID (FK → FLIGHT_INSTANCE) the scheduled flight being booked

ClassIndicator (Business / Economy) class for this booking record

TotalPrice_LocalCurrency (airport tax origin + airport tax dest + flight price; stored in local currency)

CurrencyCode (the local currency code for price)

Status (booked / canceled / scratched)

Customer_ID (FK → CUSTOMER) who is responsible for payment

AmountPaidSoFar_LocalCurrency

OutstandingBalance_LocalCurrency

BOOKING_PASSENGER

BookingPassengerID (PK)

BookingNo (FK → BOOKING)

PrintFirstName

PrintLastName

CustomerID (FK → CUSTOMER) link if passenger is a registered customer

AIRPORT_TAX

TaxID (PK)

CityID (FK→CITY) origin/destination city for which this tax applies

CurrencyCode

TaxAmount (amount in that city's local currency)

EffectiveDate (date) tax applicable from this date

CURRENCY

CurrencyCode (PK) e.g., CAD, USD, GBP, FRF, DEM, ITL

CurrencyName

EXCHANGE_RATE

RateID (PK) or composite PK (CurrencyCode, RateDate)

CurrencyCode (FK → CURRENCY)

RateDate (date)

RateToBase (e.g., conversion to a chosen base currency or to USD)

PAYMENT (optional)

PaymentID (PK)

BookingNo (FK → BOOKING)

PaymentDate

Amount_LocalCurrency

CurrencyCode

PaymentMethod

Relationships & Cardinalities (summary)

- AIRLINE - FLIGHT : 1 : M

(one airline has many flights; flight belongs to exactly one airline)

- CITY - BOOKING_OFFICE : 1 : 1

(there is a single booking office in each city)

- FLIGHT - FLIGHT_AVAILABILITY: 1 : M

(one flight can have many scheduled instances/dates)

- FLIGHT_AVAILABILITY - BOOKING : 1 : M

(many bookings can be made for the same flight instance; a booking refers to one flight instance)

- CUSTOMER - BOOKING : 1 : M (as ResponsibleCustomer)

(one customer may be responsible for many bookings)

- BOOKING - BOOKING_PASSENGER : 1 : M

(each booking may include one or more passengers)

- CUSTOMER - CUSTOMER_EMAIL : 1 : M

(a customer may have zero or more email addresses; each email unique and links to exactly one customer)

- CUSTOMER - CUSTOMER_PHONE : 1 : M

(zero or more phones per customer; phones may be shared by several customers if desired — but if phones must be shared, you would normalize PHONE as an entity and create a linking table to customers. The problem statement allows shared phone numbers.)

- CITY - AIRPORT_TAX : 1 : M (per effective date)

(airport taxes stored per city; may have multiple effective dates)

- CURRENCY - EXCHANGE_RATE : 1 : M (per date)

Q3:

Question #3: A company called **EduTemps** provides temporary teaching staff to schools throughout England. The following unnormalized table lists information about the temporary teachers, the hours they worked, and the schools they worked at.

NIN	contractNo	hours	tName	schoolID	schoolName	schoolCity
2103	T9001	10	Brown A	S301	Park School	Manchester
2198	T9001	18	Khan R	S301	Park School	Manchester
2205	T9001	20	Lewis H	S114	Green School	Birmingham
2103	T9001	12	Brown A	S114	Green School	Birmingham

- The table shown above is susceptible to update anomalies. Provide examples of insertion, deletion, and update anomalies.
- Describe and illustrate the process of normalizing the table to 3NF, by identifying the functional dependencies represented by the attributes. State any assumptions you make.

a.

1. Insertion Anomaly

We cannot insert new data unless other data already exists.

Example: Suppose a new school Blue School, Leeds registers with EduTemps, but no teacher has worked there yet. We can't add the school to the database until at least one teacher record (NIN, contractNo, etc.) exists for it.

It's a problem because we're forced to leave the new school unrecorded, because the design only allows inserting data when a teacher is assigned.

2. Deletion Anomaly

Deleting one record can cause unintended loss of important data about other entities.

Example: If the last record involving Green School (S114) is deleted (say, teacher Lewis H's contract ends), then all information about Green School (its name and city) is lost even though the school still exists and may hire teachers in the future.

It's a problem because the school's details depend on teacher records which shouldn't happen.

3. Update Anomaly

We must update multiple rows to maintain consistency, and missing one causes data inconsistency.

Example: If Park School (S301) changes its name to Parkview Academy, We'll need to update every row with schoolID S301. If one row remains unchanged, the data becomes inconsistent.

It's a problem because redundant data storage leads to inconsistency after partial updates.

b.

1. Functional Dependencies:

NIN → **tName** (Each teacher's name depends on their unique NIN)

schoolID → **schoolName, schoolCity** (Each school's name and city depend on its unique ID)

(NIN, contractNo, schoolID) → **hours** (The number of hours a teacher works at a school depends on the combination of NIN, contractNo, schoolID)

Assumption: A teacher can teach at multiple schools under one contract, or at the same school under different contracts.

2. Unnormalized (UNF) → 1NF

1NF rule: No repeating groups or multi-valued attributes. The table already has atomic values (each cell = one value),so it is in 1NF.

3. 1NF → 2NF**Teacher Table**

<u>NIN</u>	tName
2103	Brown A
2198	Khan R
2205	Lewis H

School Table

<u>schoolID</u>	schoolName	schoolCity
S301	Park School	Manchester
S114	Green School	Birmingham

Assignment Table

<u>NIN</u>	<u>contractNo</u>	<u>schoolID</u>	hours
2103	T9001	S301	10

2198	T9001	S301	18
2205	T9001	S114	20
2103	T9001	S114	12

All non-key attributes in each table depend on their entire primary key. Hence, the schema is now in **2NF**.

4. 2NF $\rightarrow$ 3NF

Checking each table:

- In Teacher, tName depends only on NIN.
- In School, schoolName and schoolCity depend only on schoolID.
- In Assignment, hours depends only on (NIN, contractNo, schoolID).

$\rightarrow$ No transitive dependencies remain.

Question#4:The casting office created the following un-normalized table, FILM_CASTING, to track casting details.

FILM_CASTING

FilmID, ActorID, FilmTitle, DirectorName, ActorName, AgentID, AgentPhone

Functional Dependencies (FDs) are given as:

1. FilmID $\rightarrow$ FilmTitle, DirectorName
2. ActorID $\rightarrow$ ActorName, AgentID
3. AgentID $\rightarrow$ AgentPhone
4. FilmID, ActorID $\rightarrow$ All other attributes (No other single attribute determines the whole record).

- A. Identify the Primary Key (PK) for the FILM_CASTING table based on the given Functional Dependencies.
- B. The table is currently in 1NF. Which is the highest Normal Form (NF) it violates (2NF or 3NF)? Justify your answer by giving an example of a dependency violation.
- C. Decompose the table to Second Normal Form (2NF). List the new tables created and state the specific dependencies that were removed.
- D. Decompose the 2NF tables further to Third Normal Form (3NF). List all final tables in their 3NF form with their PK and FK
- E. In the original FILM_CASTING table, AgentPhone is stored repeatedly for every film an actor is in. Which type of data redundancy anomaly (Insert, Update, or Delete) would this cause if the agent's phone number changes?

- a. Primary Key = (FilmID, ActorID)

(Neither FilmID nor ActorID alone determine all attributes, so the composite is the candidate key.)

- b. It violates 2NF.

The table is in 1NF (atomic attributes), but there are partial dependencies on a proper subset of the primary key:

- FilmID $\rightarrow$ FilmTitle, DirectorName (these depend on FilmID alone, not on the full key (FilmID, ActorID)).
- ActorID $\rightarrow$ ActorName, AgentID (these depend on ActorID alone).

Because non-key attributes (FilmTitle, DirectorName, ActorName, AgentID) depend on part of the composite primary key, the relation violates 2NF.

(There is also a transitive dependency ActorID $\rightarrow$ AgentID and AgentID $\rightarrow$ AgentPhone which will be addressed when moving to 3NF.)

- c. Create these tables:

1. FILM

- Attributes: FilmID (PK), FilmTitle, DirectorName
- FD removed: FilmID $\rightarrow$ FilmTitle, DirectorName (moved out of FILM_CASTING)

2. ACTOR

- Attributes: ActorID (PK), ActorName, AgentID
- FD removed: ActorID $\rightarrow$ ActorName, AgentID (moved out)

3. FILM_CASTING_2NF (the reduced assignment/join table)

- Attributes: FilmID, ActorID
- Primary Key: (FilmID, ActorID)
- This table records which actor is cast in which film (no repeated film/actor descriptive attributes).

After this decomposition there are no partial dependencies, the join table has only the composite key. However, a transitive dependency remains: ActorID $\rightarrow$ AgentID and AgentID $\rightarrow$ AgentPhone implies ActorID $\rightarrow$ AgentPhone transitively if AgentPhone were still placed with Actor or in any table where it can repeat.

- d.

From 2NF tables, we still have AgentID $\rightarrow$ AgentPhone. To eliminate transitive dependency (ActorID $\rightarrow$ AgentID $\rightarrow$ AgentPhone), create a separate AGENT table.

Final 3NF tables:

1. FILM
 - Columns: FilmID (PK), FilmTitle, DirectorName
 - FD: FilmID $\rightarrow$ FilmTitle, DirectorName
2. ACTOR
 - Columns: ActorID (PK), ActorName, AgentID (FK $\rightarrow$ AGENT.AgentID)
 - FD: ActorID $\rightarrow$ ActorName, AgentID
3. AGENT
 - Columns: AgentID (PK), AgentPhone
 - FD: AgentID $\rightarrow$ AgentPhone
4. FILM_CASTING
 - Columns: FilmID, ActorID
 - Primary Key: (FilmID, ActorID)
 - Foreign Keys: FilmID $\rightarrow$ FILM(FilmID), ActorID $\rightarrow$ ACTOR(ActorID)
 - This table contains no non-key descriptive attributes — it links films and actors.

e.

Because AgentPhone was originally stored repeatedly for every film an actor is in, changing the agent's phone number would require updating that phone number in every row where it appears.

This is an Update Anomaly.

Example: Agent A123 has a phone 0123-456. If that number changes to 0987-654, you must update all FILM_CASTING rows that include actors with AgentID = A123. If you forget to update some rows, the database becomes inconsistent (some rows show old phone, some new).

(Also: storing AgentPhone redundantly could cause insertion/deletion anomalies too e.g., deleting the only casting record for an actor could remove the only copy of that agent phone, but the direct effect of a phone number change is an update anomaly.)

Question#5: Examine the **City College Enrollment Form** shown below:

City College ENROLLMENT FORM				
Student ID: <u>S1001</u>		First Name: <u>James</u>		
Last Name: <u>Smith</u>		Address: <u>123 Oak Street</u>		
Course ID	Course Name	Credits	Instructor Name	Enrollment Date
CS101	Introduction to Computer Science	3	Dr. Baker	15/01/2024
MA202	Calculus II	4	Dr. Johnson	15/01/2024
PH301	Physics	3	Dr. Lee	15/01/2024

1. Identify the **functional dependencies** represented by the attributes shown in the form above. State any assumptions that you make about the data and the attributes.
2. Describe and illustrate the process of **normalizing** the attributes shown in the form above to produce a set of well-designed **3NF (Third Normal Form)** relations.
3. Identify the **primary**, **alternate**, and **foreign keys** in your 3NF relations.

1. Assumptions:

- StudentID uniquely identifies a student (one student record per StudentID).
- CourseID uniquely identifies a course (course code → course name & credits).
- Each CourseID has one current instructor (i.e., CourseID → InstructorName). If courses can have multiple instructors/offerings, we would model offerings separately but the form shows a single instructor per course, so we assume that.
- EnrollmentDate is the date a particular student enrolled in that course; therefore it depends on the student–course pair.
- Instructor names are unique enough for this exercise (or we can introduce InstructorID. I'll include Instructor as a separate entity with InstructorID to be robust).
- No other hidden attributes (e.g., semester) unless noted.

Functional Dependencies (FDs)

From the form and assumptions:

1. StudentID → FirstName, LastName, Address
2. CourseID → CourseName, Credits, InstructorID (we introduce InstructorID)
3. InstructorID → InstructorName
4. (StudentID, CourseID) → EnrollmentDate (the enrollment date is for a student in a specific course)

Notes:

- StudentID and CourseID together determine the enrollment tuple (like hours in previous problems).
- No single attribute (StudentID or CourseID alone) determines EnrollmentDate.

2.

This table already has atomic values: meaning no repeating groups or multivalued attributes so it satisfies 1NF.

However, for performance and simplicity, we can optionally decompose the EnrollmentDate attribute into three separate attributes:

EnrollmentDay, EnrollmentMonth, EnrollmentYear

This is practical solution because of:

Query efficiency: Searching or filtering by month, year, or specific date part (e.g., “all enrollments in January 2024”) becomes faster without needing string/date parsing functions.

Lower time complexity for operations like:

- `SELECT * FROM Enrollment WHERE EnrollmentYear = 2024;`
- `COUNT(*)` of enrollments by month.

2NF remove partial dependencies

Primary key for the enrollment-like relation is (StudentID, CourseID) because enrollment is per student per course.

Partial dependencies that violate 2NF:

- StudentID → FirstName, LastName, Address (depends only on StudentID, part of composite key)
- CourseID → CourseName, Credits, InstructorID (depends only on CourseID)

Decompose into:

- STUDENT(StudentID, FirstName, LastName, Address) - PK: StudentID
- COURSE(CourseID, CourseName, Credits, InstructorID) - PK: CourseID
- ENROLLMENT(StudentID, CourseID, EnrollmentDate) - PK: (StudentID, CourseID)

After this, ENROLLMENT has only the composite key + EnrollmentDate (no partial dependencies) so 2NF is satisfied.

3NF remove transitive dependencies

Check for transitive dependencies: CourseID $\rightarrow$ InstructorID and InstructorID $\rightarrow$ InstructorName would imply CourseID $\rightarrow$ InstructorName transitively. To place instructor attributes only once, separate Instructor:

Decompose into:

- INSTRUCTOR(InstructorID, InstructorName) — PK: InstructorID
- Remove InstructorName from COURSE and keep only InstructorID as FK

Now all non-key attributes depend only on the key of their table and not on other non-key attributes. This satisfies 3NF.

3. Final 3NF Relations (with keys)

1. STUDENT

- Attributes: StudentID (PK), FirstName, LastName, Address
- PK: StudentID
- Alternate keys: none given (could be e.g., email if available)

2. INSTRUCTOR

- Attributes: InstructorID (PK), InstructorName
- PK: InstructorID
- (If instructor phone/email existed, it would go here too.)

3. COURSE

- Attributes: CourseID (PK), CourseName, Credits, InstructorID (FK $\rightarrow$ INSTRUCTOR.InstructorID)
- PK: CourseID
- FK: InstructorID references INSTRUCTOR(InstructorID)

4. ENROLLMENT (student–course join)

- Attributes: StudentID (FK $\rightarrow$ STUDENT.StudentID), CourseID (FK $\rightarrow$ COURSE.CourseID), EnrollmentDate
- PK: composite (StudentID, CourseID)

- FKs: StudentID $\rightarrow$ STUDENT, CourseID $\rightarrow$ COURSE

Primary keys (PK):

- STUDENT: StudentID
- INSTRUCTOR: InstructorID
- COURSE: CourseID
- ENROLLMENT: (StudentID, CourseID)

Foreign keys (FK):

- COURSE.InstructorID $\rightarrow$ INSTRUCTOR.InstructorID
- ENROLLMENT.StudentID $\rightarrow$ STUDENT.StudentID
- ENROLLMENT.CourseID $\rightarrow$ COURSE.CourseID

Alternate keys: none shown in the form.